

IMPRINT

HH GOA 2026 • TASK 3

TEAM BUILD PLAN • 11:45 AM → 9:00 PM

KHUSHI + SAMIDHA + DISHA

FACE → BIOMETRIC SEARCH → WEB EVIDENCE → PROVENANCE → BLOCKCHAIN

Mission: finish a genuinely working, record-ready system by **7:00 PM**. 7:00–9:00 PM is protected buffer for integration, recording, upload and submission.

SYSTEM WE ARE BUILDING

A real biometric evidence pipeline: an input face is converted to an embedding; a 1:N local biometric index retrieves top-K candidates; a genuine Google Lens search independently discovers web/social evidence; candidates are fused and verified; provenance is frozen into canonical JSON; SHA-256 creates the fingerprint; Base Sepolia anchors it; the transaction is read back and verified.

NON-NEGOTIABLE: The local 1:N biometric search is the core engine. Google Lens is a genuine evidence-discovery layer used to satisfy the task requirement and connect the local candidate to a real social/web post.

DEFINITION OF DONE

Gate	Must be true by 7 PM
1. Face	Input image produces exactly one usable face template or fails safely.
2. 1:N	FAISS index returns ranked top-K candidate identities from a real enrolled corpus.
3. Web	Google Lens/SerpApi performs a real search and finds at least one real social/web candidate.
4. Fusion	System records biometric score + web provenance and selects an evidence candidate.
5. Provenance	Canonical evidence JSON is created deterministically.
6. Blockchain	SHA-256 fingerprint is written to Base Sepolia.
7. Verification	Transaction is read back; chain/payload/hash are checked and VERIFIED.
8. Demo	Exact final command runs successfully without manual code edits.

1. TEAM OWNERSHIP — WHO OWNS WHAT

All 3 people work on backend until the first integration checkpoint. After that, each person owns a subsystem, but everyone helps with debugging.

Person	Primary owner	Secondary responsibility	Critical deliverable
KHUSHI	Biometric core + orchestration	Local UI + final integration	YuNet/SFace → FAISS → candidate API → full pipeline runner
SAMIDHA	Web discovery + candidate retrieval	UI/UX + final recording	Google Lens provider → social result extraction → candidate media/provenance
DISHA	Evidence + blockchain + verification	Tests + README + secrets audit	Canonical JSON → SHA-256 → Base Sepolia tx → readback verifier

Merge rule: Khushi owns final branch integration. Samidha and Disha do not simultaneously rewrite the same core orchestration files after 1:00 PM; they commit clean modules and Khushi integrates.

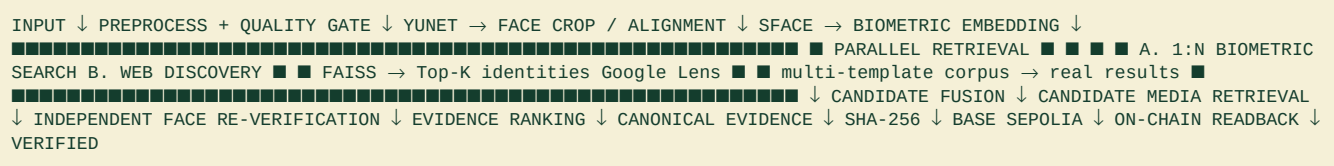
2. 11:45 AM → 1:00 PM: THE FIRST WORKING SYSTEM

Goal: at 1:00 PM, the three of you must be able to execute one end-to-end path, even if it is ugly. Do not build UI during this phase.

Time	KHUSHI	SAMIDHA	DISHA
11:45–12:00	Create project structure; load YuNet/SFace; define `FaceTemplate` + `Candidate` data models.	Install/test SerpApi client; verify real image upload → Lens request on the demo image.	Create `.env`, `.env.example`, `.gitignore`; verify wallet/RPC connection to Base Sepolia.
12:00–12:25	Implement face detection, crop/alignment, quality gate, SFace embedding.	Parse Lens results into normalized candidate objects: URL, title, image URL, domain, result type.	Implement SHA-256 helper + canonical JSON serialization.
12:25–12:45	Build FAISS index from enrolled corpus; implement `search_top_k(embedding,k)`.	Implement candidate downloader with timeout/type/size checks; social-domain classifier.	Implement Base transaction payload encoder + send function.
12:45–1:00	Write first orchestrator: input → embedding → FAISS top-K → call web search → pass candidates forward.	Return first real Lens/social result into the orchestrator.	Read transaction back + extract payload; prove local hash == on-chain hash on a test string.

1:00 PM CHECKPOINT: If each component works separately but integration does not, stop adding features. The next 60 minutes are integration-only.

3. TARGET ARCHITECTURE — IMPLEMENTATION ORDER



The important architectural difference is that **1:N biometric retrieval is a first-class retrieval engine**, not a final score. The web layer is a second, independent source of evidence. The two are joined by candidate fusion and provenance.

Minimum viable data structures

```
Identity: id display_name image_refs[] embedding_refs[] metadata BiometricCandidate: identity_id similarity
matched_template_id quality WebCandidate: provider result_type source_url image_url title domain retrieved_at Evidence:
schema input_hash biometric_candidates web_candidates selected_candidate verification provenance
```

4. 1:00 PM → 7:00 PM: BUILD HARD, THEN FREEZE

Time	Primary goal	Khushi	Samidha	Disha	Exit criteria
1:00–1:30	Integration Sprint 1	Wire FAISS top-K into orchestrator.	Wire Lens candidates into same pipeline.	Wire hash + chain module into orchestrator.	Single command reaches evidence object.
1:30–2:00	Real candidate flow	Ensure top-K has real faces + stable IDs.	Find real social/web result from live Lens search.	Generate canonical evidence + local fingerprint.	One real candidate is selected and hashed.
2:00–2:45	Fusion + ranking	Implement candidate dedupe by URL/image identity and biometric rank.	Add domain/source metadata and retrieve accessible media.	Add evidence schema + score fields.	Top candidates are ranked; best candidate is reproducible.
2:45–3:30	Robustness pass	No-face / multi-face / low-quality handling.	Lens timeout / no-result / inaccessible-media handling.	Blockchain retry / receipt / readback error handling.	Pipeline fails gracefully on controlled bad cases.
3:30–4:15	Core proof run	Run final probe against enrolled corpus.	Run genuine Lens search and confirm social/web result.	Write fingerprint to Base Sepolia + read it back.	FULL END-TO-END SUCCESS #1.
4:15–5:00	Evidence + UX	Expose top-K, selected candidate and similarity gap.	Expose source URL, retrieval path and search status.	Expose evidence JSON, SHA-256, tx hash and verification status.	Judge can understand every stage from one screen.
5:00–5:45	UI integration	Connect live backend state to local UI.	Hacker House visual system: green/yellow/pink/cream + concise status cards.	Verification panel + technical metadata + error states.	UI never lies; statuses come from backend.
5:45–6:15	Second proof run	Run the complete pipeline from clean start.	Confirm the web result is live, not hardcoded/cached artifact.	Confirm a fresh on-chain tx and exact readback.	FULL END-TO-END SUCCESS #2.
6:15–6:30	Security / repository	Remove debug code and temporary files.	Remove accidental hardcoded results.	Secret audit; <code>`.env`</code> excluded; README commands.	Repo safe to publish.
6:30–7:00	FREEZE + REHEARSAL	Own final integration; tag release candidate.	Prepare recording scene order.	Prepare README + final verification checklist.	Exact recording command succeeds.

7:00 PM = FEATURE FREEZE. From this moment, only blockers may be fixed. No new frameworks, no new model swaps, no new architecture, no redesign.

5. KHUSHI — BIOMETRIC ENGINE + ORCHESTRATION

Build this in the order below. The biometric engine must remain usable independently of the web layer.

Module	Implementation target	Definition of done
Face detector	YuNet model; detect bbox + confidence.	Returns face(s) and confidence; handles zero/multiple faces.
Quality gate	bbox size, detector confidence, simple blur metric.	Rejects obviously unusable inputs.
Alignment	Normalize crop for consistent SFace embedding.	Same normalization applied to probe and corpus images.
Embedding	SFace embedding vector.	One stable vector per template.
Enrollment	Process corpus images into identities + multiple templates.	Can rebuild index from corpus without manual edits.
1:N index	FAISS index + mapping vector position → identity/template.	`search_top_k()` returns ranked candidates.
Candidate fusion	Merge biometric candidate IDs with web candidates when evidence links them.	One normalized candidate schema.
Orchestrator	Single pipeline command with machine-readable status events.	End-to-end path can run from a clean terminal.

Do not spend time building a giant corpus today. Build the architecture for a large corpus, but use a verified, demonstrable demo corpus now. The future scale-up is an index/data problem, not a rewrite of the pipeline.

6. SAMIDHA — GENUINE WEB DISCOVERY + MEDIA RETRIEVAL

Module	Implementation target	Definition of done
Image upload	Send the actual local image (or derived search image) to the visual-search provider.	Provider returns a live search identifier/result.
Lens search	Request visual/exact matches with fresh-search behavior where supported.	Raw search response saved locally for debugging/evidence.
Result normalization	Extract result URL, title, domain, image URL, result type and timestamp.	All candidate results use one common schema.
Social filter	Recognize real domains: Facebook, Instagram, X, LinkedIn, YouTube, etc.	At least one live social/web candidate is surfaced in demo.
Candidate media	Download accessible images safely; do not depend on page scraping.	Candidate image can be re-verified locally.
Fallback	If Lens returns weak/no social result, use a targeted web-search fallback or an alternate result from the same live search response.	System reports fallback explicitly; never invents a source.
UI search panel	Show provider, result count, selected source and URL.	Judge can see that discovery actually happened.

Important: Never hard-code the winning post URL. The recorded demo must show the live search step that produces the result.

7. DISHA — PROVENANCE + SHA-256 + BASE SEPOLIA

Module	Implementation target	Definition of done
Canonicalization	Deterministic JSON: sorted keys, stable encoding, explicit schema/version.	Same evidence object always produces same hash.
Fingerprint	SHA-256 over canonical evidence bytes.	64-character hexadecimal fingerprint generated.
Payload	Encode schema marker + version + 32-byte hash (or agreed transaction-data format).	Decoder can recover exact expected hash.
Blockchain	Base Sepolia testnet; real signed transaction.	Receipt/status available.
Readback	Fetch transaction and decode payload.	Chain ID, tx success and payload verified.
Integrity check	Compare recovered on-chain hash to local hash.	Explicit MATCH / VERIFIED result.
Artifacts	Save evidence JSON, canonical JSON, fingerprint, raw search, receipt.	Reproducible local evidence package.

Privacy rule: Never put raw faces, embeddings, API keys or wallet private keys on-chain or into the public repository.

8. UI — ONLY WHAT THE JUDGE NEEDS TO SEE

One local page. No login. No deployment. No unnecessary dashboard. It should visualize the actual backend state.

```
IMPRINT FACE → BIOMETRIC SEARCH → WEB EVIDENCE → CHAIN [INPUT FACE] [BIOMETRIC SEARCH] face preview 1:N search quality
✓ Top 3 candidates embedding ✓ #1 0.91 #2 0.84 #3 0.76 [WEB DISCOVERY] Google Lens ✓ 27 results 3 social candidates
Selected: URL: ... [EVIDENCE] Selected candidate face similarity source retrieved_at evidence hash [BLOCKCHAIN] Base
Sepolia ✓ TX: 0x... Block: ... Hash match ✓ EVIDENCE VERIFIED
```

UI principle: **every green check must come from the actual backend result**. Never show “VERIFIED” as decorative text before verification completes.

9. 7:00 PM → 9:00 PM: RECORDING + SUBMISSION BUFFER

Time	Action	Owner	Backup
7:00–7:15	Clean restart + final rehearsal	All 3	Khushi
7:15–7:45	Record continuous end-to-end demo	Samidha	Khushi
7:45–8:00	Watch recording once for visible failure / missing step	All 3	Disha
8:00–8:15	Upload video and verify link from another browser/tab	Samidha	Disha
8:15–8:30	Final GitHub push, README, repo sanity check	Disha	Khushi
8:30–8:45	Submission form with final repo + video links	Disha	Samidha
8:45–9:00	Reserve for emergency fixes / resubmission checks	All 3	All 3

10. EXACT RECORDING SEQUENCE

Step	Screen action	What must visibly happen
1	Launch IMPRINT	Clean app starts; no debug setup.
2	Load input face	Face detected + quality PASS.
3	Run 1:N search	Top-K identities appear with scores.
4	Run genuine web discovery	Search request/results happen live; social candidate appears.
5	Candidate fusion	Selected candidate + biometric/web evidence shown.
6	Generate evidence	Canonical JSON + SHA-256 appear.
7	Anchor on blockchain	Base Sepolia tx submitted; tx hash shown.
8	Read back	Actual tx payload decoded and hash compared.
9	Final state	ON-CHAIN VERIFIED / EVIDENCE VERIFIED.

Keep the recording unedited. Do not refresh into a pre-populated success state. The visible sequence should show the system actually moving through the pipeline.

11. FINAL PRE-SUBMISSION CHECKLIST

CHECK	STATUS
YuNet detects a usable face	■
SFace creates embedding	■
FAISS 1:N search returns ranked candidates	■
Google Lens performs genuine live search	■
At least one real social/web result is discovered	■
Candidate is independently face-verified	■
Evidence record is deterministic	■
SHA-256 is generated from canonical evidence	■
Base Sepolia transaction is real and successful	■
Transaction is read back and payload decoded	■
Local and on-chain hashes match	■
No secrets/hardcoded result leaked to repo	■
README has exact setup/run/blockchain/limitations	■
Clean final end-to-end run completed	■
Unedited video link verified	■
GitHub repo link verified	■
Submission form submitted before 11:59 PM	■

12. EMERGENCY RULES

- At 1:00 PM: no integration → stop feature work and make the simplest end-to-end path work.
- At 3:30 PM: blockchain unstable → use the simplest real Base Sepolia transaction path; do not deploy a complex contract.
- At 4:30 PM: web result unstable → use another genuine result from the live search or documented fallback; never hard-code.
- At 6:30 PM: UI unfinished → ship a minimal functional UI and prioritize the pipeline.
- At 7:00 PM: freeze features. The goal becomes a successful recording and submission, not a prettier system.

FINAL TEAM MANTRA

Build the engine first. Prove every dependency live. Freeze by 7. Submit by 9.

This plan intentionally keeps 1:N biometric retrieval as the architectural core while preserving the task's required genuine web/social search and blockchain verification.